PropOS 开发与 UI 设计同步指南

> **版本**: v2.0
>
> **日期**: 2026-04-09
>
> **适用项目**: PropOS Phase 1
>
> **适用角色**: 开发者 + UI/UX 设计师
>
> **技术栈**: Flutter（移动端）+ Vue 3 + Element Plus（PC Admin）

目录

1. 核心理念
    - 1.1 问题：传统流程的痛点
    - 1.2 PropOS 方案："骨架先行，并行打磨"
2. 整体工作流
3. 分阶段执行计划
    - Phase A：骨架生成（Day 0~2）
    - Phase B：对齐日（Day 3）
    - Phase C：双线并行（Day 4~10）
    - Phase D：视觉接入（Day 11~14）
4. 角色职责分工
    - 4.1 开发者职责
    - 4.2 设计师职责
    - 4.3 禁止事项
5. 设计师输入物清单
    - 5.1 截图包（按页面编号）
    - 5.2 业务色彩语义定义
    - 5.3 数据边界速查表
6. Design Token 规范
    - 6.1 颜色 Token
    - 6.2 字体 Token
    - 6.3 间距 Token
    - 6.4 圆角 Token
7. Material 3 主题 / Element Plus 主题对接规范
    - 7.1 Flutter 端（Material 3）
    - 7.2 admin 端（Element Plus）

8. 页面清单与优先级
P0：第一周必须完成（验收基础）
P1：第二周完成
P2：第三周完成
9. 协作检查清单
9.1 开始骨架开发前（Day 0）
9.2 对齐日前（Day 3）
9.3 Design Token 接入前（Day 11）
9.4 视觉验收（Day 14）
附录：快速参考
状态色彩速查
Mock 数据替换路径

---

1. 核心理念

1.1 问题：传统流程的痛点

传统"设计先行"流程：

```
① 需求文档 → ② 设计师出 Figma 稿 → ③ 开发实现 → ④ 返工对齐
```

痛点：

- 设计师对数据密度没有感知（"这个列表一行到底几个字段？"）
- 开发实现后发现布局问题，两端同时返工
- 设计与代码长期存在偏差，无专人维护 Design Token

1.2 PropOS 方案："骨架先行，并行打磨"

```
① 开发生成 Flutter / admin 页面骨架（MockData）
          ↓
② 真机/浏览器截图 + 录屏 → 设计师看真实数据密度
          ↓
③ 开发 + 设计双线并行推进
          ↓
⑤ 设计师输出 Design Token → 开发写入 Material 3 ThemeData / Element Plus 主题
          ↓
⑤ 视觉精修：按 Figma 稿调整间距、色彩、字体
```

核心原则：

- **骨架即生产代码**，不存在"原型→推倒重来"

- **MockData** 可随时替换为真实 API，不影响页面逻辑
- **Design Token** 驱动，设计更改只改 Material 3 ThemeData / Element Plus 主题配置，不改组件结构

---

2. 整体工作流

```
┌──────────────────────────────────────────────────────────────┐
│  第 0 天：启动准备                                            │
│  开发者：环境搭建 + pubspec.yaml / package.json + 路由骨架      │
│  设计师：阅读 PRD.md + ARCH.md §4（路由结构）+ 竞品参考         │
└──────────────────────────────────────────────────────────────┘
                              │
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  第 1~2 天：骨架生成                                          │
│  开发者：生成全部页面 Widget / Vue 组件骨架 + MockData            │
│  设计师：整理色彩参考、字体偏好、业务风格定义（科技感/严肃感等）   │
└──────────────────────────────────────────────────────────────┘
                              │
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  第 3 天：对齐日                                              │
│  开发者：真机运行 Flutter + 浏览器运行 admin，录制操作录屏       │
│  设计师：看录屏，标注疑问："这里要不要分页？""这个状态色块太小了"  │
│  联合：30 分钟站会，统一疑问 → 形成设计约束文档（本文附录）       │
└──────────────────────────────────────────────────────────────┘
                              │
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  第 4~10 天：双线并行                                         │
│  开发者：继续完善业务逻辑、BLoC/Cubit 状态管理、错误处理         │
│  设计师：基于截图在 Figma 做视觉设计，输出 Design Token 表      │
└──────────────────────────────────────────────────────────────┘
                              │
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  第 11~14 天：视觉接入                                        │
│  开发者：将 Design Token 写入 Material 3 ThemeData / Element Plus 主题 │
│  设计师：验收视觉效果，输出标注修正（Redline）                   │
└──────────────────────────────────────────────────────────────┘
```

---

3. 分阶段执行计划

Phase A：骨架生成（Day 0~2）

开发者交付物：

文件	说明
`flutter_app/pubspec.yaml`	Flutter 依赖声明（flutter*bloc*、*dio*、*go*router、get_it 等）

`admin/package.json`	Admin 依赖声明（element-plus、axios、pinia、vue-router 等）
`flutter_app/lib/core/router/app_router.dart`	go_router 路由配置
`admin/src/router/index.ts`	Vue Router 4 完整路由树
`flutter_app/lib/shared/mock/`	各模块 MockData（覆盖所有页面所需字段）
`flutter_app/lib/core/theme/app_theme.dart` + `admin/src/styles/variables.scss`	临时占位主题变量
所有 `*.vue` 页面文件	骨架页面，含正确的布局结构和 MockData 展示

MockData 覆盖范围：

- `MockDashboardData`：NOI、EGI、OpEx、出租率、WALE（按三业态）
- `MockBuildingData`：3 栋楼 × 多楼层 × 单元列表（含状态分布）
- `MockContractData`：10 份合同（覆盖全部状态机状态）
- `MockInvoiceData`：20 条账单（覆盖 pending/paid/overdue）
- `MockWorkOrderData`：8 条工单（覆盖全部状态）
- `MockSubleaseData`：二房东视角的子租赁列表

Phase B：对齐日（Day 3）

开发者输出（给设计师）：

1. Flutter 真机截图（iPhone 14 Pro 尺寸）+ admin 浏览器截图（1920×1080）
2. 核心流程操作录屏：
   - 登录 → Dashboard 总览
   - 资产 → 楼栋 → 楼层热区图 → 单元详情
   - 合同列表 → 合同详情 → 租金递增配置
   - 财务 → 账单列表 → 核销操作
   - 工单提报全流程
1. 本文 §8 页面清单（标注"高优"页面）

会议议程（30 分钟）：

时间	内容
0~10 分钟	开发者演示骨架 App + Admin 操作流程
10~20 分钟	设计师提问：数据密度、交互边界、色块语义
20~30 分钟	联合确认：状态色彩语义、字体大小底线、卡片信息层级

Phase C：双线并行（Day 4~10）

开发者任务：

- 完善 BLoC/Cubit 状态管理（四态：initial / loading / loaded / error）
- 接入 go_router 路由守卫（redirect 检查认证、RBAC 角色判断）
- 完善表单页面的验证逻辑
- 处理空状态、加载状态、错误状态三种 UI 状态

设计师任务：

优先级	Figma 页面	输出要求
P0	Dashboard 总览	完整高保真，含数据可视化组件样式
P0	楼层平面图热区	四种状态色定义 + 悬浮弹窗样式
P1	合同详情 + 递增配置器	复杂表单布局
P1	账单列表 + 核销弹窗	列表行设计
P2	工单流程（提报/审核/完工）	移动优先布局
P3	二房东填报 Portal	独立简洁风格，与主 App 视觉上有轻微差异

Phase D：视觉接入（Day 11~14）

1. 设计师提交 Design Token 表（见 §6）
2. 开发者将 Token 写入 Material 3 ThemeData / Element Plus 主题
3. 逐页核对 Figma 标注，调整 `padding` / `border-radius` / `font-size`
4. 设计师在真机 + 浏览器上验收，提交 Redline 修正清单
5. 开发者修正后，进入功能测试阶段

---

4. 角色职责分工

4.1 开发者职责

职责	说明
主导目录结构	严格按 copilot-instructions.md 中定义的 flutter_app/lib 和 admin/src 目录结构建立
维护 MockData	MockData 字段须与真实 API 响应结构一致，便于后续替换
定义组件接口	在生成骨架时确定 Widget / Vue 组件 props 签名，设计师不得要求修改 prop 名
接入 Design Token	将设计师的 Token 表翻译为 Material 3 ThemeData / SCSS，设计师不直接改代码
性能基线	Flutter 列表使用 ListView.builder 懒加载，图片使用 cached*network*image

4.2 设计师职责

职责	说明
了解数据边界	必须知道每个字段的最大长度（如合同编号最长 100 字符）
respect 业务状态	状态色块（🟢🟡🔴⚪）是业务约束，不可仅凭美观更改颜色语义
输出 Design Token	以表格形式输出（见 §6），不直接修改代码
标注单位	Flutter 使用逻辑像素（dp），admin 使用 px
验收使用真机/浏览器	不以 Figma 预览验收，必须在真机 + 浏览器上确认

4.3 禁止事项

禁止行为	原因
设计师直接修改 Vue 代码	破坏代码分层约定
开发者绕过 Figma 自行"美化"	产生设计漂移，难以维护
设计师更改状态色彩语义	`expiring_soon` 必须是黄色系，这是业务约束
使用 Figma 截图验收替代真机	Figma 不模拟真实屏幕密度和字体渲染

---

5. 设计师输入物清单

开发者在 Day 3 交给设计师的标准输入包：

5.1 截图包（按页面编号）

```
screenshots/
├── 01_login.png
├── 02_dashboard.png
├── 03_dashboard_noi_detail.png
├── 04_dashboard_wale_detail.png
├── 05_dashboard_kpi.png
├── 06_assets_overview.png
├── 07_building_detail.png
├── 08_floor_map.png              ← 热区色块核心页
├── 09_unit_detail.png
├── 10_contract_list.png
├── 11_contract_detail.png
├── 12_escalation_config.png
├── 13_finance_overview.png
├── 14_invoice_list.png
├── 15_workorder_list.png
├── 16_workorder_new.png
├── 17_workorder_detail.png
├── 18_sublease_portal.png        ← 二房东独立入口
└── 19_settings.png
```

5.2 业务色彩语义定义

语义	含义	说明
`status.leased`	🟢 已租	必须是绿色系
`status.expiring_soon`	🟡 即将到期（≤90天）	必须是黄/橙色系
`status.vacant`	🔴 空置	必须是红色系
`status.non_leasable`	⚪ 非可租	中性灰色
`invoice.overdue`	🔴 逾期账单	与空置同语义色系
`invoice.paid`	🟢 已核销	与已租同语义色系
`alert.warning`	🟡 预警通知	与即将到期同语义色系

5.3 数据边界速查表

字段	最大长度	说明
楼栋名称	10 字符	如"A座写字楼"
合同编号	100 字符	系统生成，不会换行
租客名称	200 字符	企业名可能很长，须处理溢出
单元编号	50 字符	如"A-1501"
工单描述	无限制（TEXT）	列表页需要截断显示
金额	最大 12 位整数 + 2 位小数	须考虑对齐方式（右对齐）

---

6. Design Token 规范

设计师完成 Figma 后，以如下格式提交 Design Token 表格（Excel 或 Markdown）：

6.1 颜色 Token

Token 名称	Hex 值	使用场景
`--color-primary`	`#1976D2`	主品牌色、按钮、链接
`--color-primary-light`	`#BBDEFB`	主色容器背景
`--color-success`	`#388E3C`	成功/已租状态
`--color-danger`	`#D32F2F`	错误/空置/逾期
`--color-warning`	`#F57C00`	预警/即将到期
`--color-surface`	`#FAFAFA`	卡片背景
`--color-background`	`#F5F5F5`	页面背景
`--color-text-primary`	`#212121`	主文字色
`--color-text-secondary`	`#757575`	次要文字色

> 注意：Token 名称由开发者定义，设计师填入对应的 Hex 值。Token 名称不可更改。

6.2 字体 Token

Token 名称	字号	字重	用途
`--font-display-large`	32px	Bold	大看板数字（NOI、出租率）
`--font-display-medium`	24px	Bold	次级看板数字
`--font-headline-large`	20px	SemiBold	页面标题
`--font-headline-medium`	16px	SemiBold	卡片标题
`--font-body-large`	16px	Regular	正文
`--font-body-medium`	14px	Regular	列表行内容
`--font-label-large`	14px	Medium	按钮文字
`--font-label-small`	11px	Regular	徽标、状态标签

6.3 间距 Token

Token 名称	值	使用场景
`--spacing-xs`	4px	图标与文字间距
`--spacing-sm`	8px	组件内部间距
`--spacing-md`	16px	卡片 padding、列表行间距
`--spacing-lg`	24px	页面水平边距
`--spacing-xl`	32px	页面顶部间距

6.4 圆角 Token

Token 名称	值	使用场景
`--radius-sm`	4px	小标签、徽标
`--radius-md`	8px	输入框
`--radius-lg`	12px	卡片
`--radius-xl`	16px	底部弹窗、对话框

7. Material 3 主题 / Element Plus 主题对接规范

7.1 Flutter 端（Material 3）

开发者依据 §6 的 Token 表，在 `flutter_app/lib/core/theme/app_theme.dart` 中配置 Material 3 主题：

```
// flutter_app/lib/core/theme/app_theme.dart
import 'package:flutter/material.dart';

class AppTheme {
  static ThemeData light() => ThemeData(
    useMaterial3: true,
    colorScheme: const ColorScheme.light(
      primary: Color(0xFF1976D2),
      onPrimary: Colors.white,
      secondary: Color(0xFF388E3C),
      tertiary: Color(0xFFF57C00),     // warning 语义色
      error: Color(0xFFD32F2F),
      surface: Color(0xFFFAFAFA),
      outline: Color(0xFF9E9E9E),      // non_leasable 语义色
    ),
    textTheme: const TextTheme(
      displayLarge: TextStyle(fontSize: 32),
      displayMedium: TextStyle(fontSize: 24),
      headlineLarge: TextStyle(fontSize: 20),
      headlineMedium: TextStyle(fontSize: 16),
      bodyLarge: TextStyle(fontSize: 16),
      bodyMedium: TextStyle(fontSize: 14),
      labelLarge: TextStyle(fontSize: 14),
      labelSmall: TextStyle(fontSize: 11),
    ),
    extensions: const <ThemeExtension<dynamic>>[
      CustomColors(
        success: Color(0xFF388E3C),
        warning: Color(0xFFF57C00),
      ),
    ],
  );
}

/// 自定义语义色扩展
@immutable
class CustomColors extends ThemeExtension<CustomColors> {
  final Color success;
  final Color warning;
  const CustomColors({required this.success, required this.warning});
  // copyWith / lerp 省略
}
```

业务语义色映射（固定，设计师不可更改）：

```
// flutter_app/lib/shared/utils/status_color.dart
Color unitStatusColor(BuildContext context, String status) {
```

```
  final scheme = Theme.of(context).colorScheme;
  final custom = Theme.of(context).extension<CustomColors>()!;
  return switch (status) {
    'leased'        => custom.success,
    'expiring_soon' => custom.warning,
    'vacant'        => scheme.error,
    'non_leasable'  => scheme.outline,
    _               => scheme.onSurface,
  };
}
```

7.2 admin 端（Element Plus）

开发者在 `admin/src/styles/variables.scss` 中覆盖 Element Plus 主题变量：

```
/* admin/src/styles/variables.scss */

/* Element Plus 主题色覆盖 */
:root {
  --el-color-primary: #1976D2;
  --el-color-success: #388E3C;
  --el-color-warning: #F57C00;
  --el-color-danger: #D32F2F;
  --el-color-info: #9E9E9E;
}

/* 自定义 Token（与 Flutter 端保持一致） */
:root {
  --color-surface: #FAFAFA;
  --color-background: #F5F5F5;
  --color-text-primary: #212121;
  --color-text-secondary: #757575;

  --spacing-xs: 4px;
  --spacing-sm: 8px;
  --spacing-md: 16px;
  --spacing-lg: 24px;
  --spacing-xl: 32px;

  --radius-sm: 4px;
  --radius-md: 8px;
  --radius-lg: 12px;
  --radius-xl: 16px;
}
```

admin 端状态色使用 Element Plus 的 `type` 属性（开发者在组件中使用）：

```
<!-- admin 端状态 Tag 示例 -->
<el-tag type="success">已租</el-tag>
<el-tag type="warning">即将到期</el-tag>
<el-tag type="danger">空置</el-tag>
<el-tag type="info">非可租</el-tag>
```

设计师只需修改 **ThemeData** 中的色彩值和 **Element Plus CSS** 变量，90% 的视觉变更会自动传导到全 App + Admin。

---

8. 页面清单与优先级

P0：第一周必须完成（验收基础）

编号	页面	Flutter 路由	admin 路由	设计复杂度
01	登录页	`/login`	`/login`	★☆☆
02	Dashboard 总览	`/dashboard`	`/dashboard`	★★★
03	资产概览	`/assets`	`/assets`	★★☆
04	楼层平面图（热区）	`/assets/buildings/:bid/floors/:fid`	`/assets/building/:id/floor/:id`	★★★
05	单元详情	`/assets/units/:id`	`/assets/unit/:id`	★★☆
06	合同列表	`/contracts`	`/contracts`	★☆☆
07	合同详情	`/contracts/:id`	`/contracts/:id`	★★★

P1：第二周完成

编号	页面	Flutter 路由	admin 路由	设计复杂度
08	财务概览（NOI 看板）	`/finance`	`/finance`	★★★
09	账单列表	`/finance/invoices`	`/finance/invoices`	★★☆
10	工单列表	`/workorders`	`/workorders`	★★☆
11	工单提报	`/workorders/new`	`/workorders/new`	★★☆
12	KPI 看板	`/finance/kpi`	`/dashboard/kpi`	★★★
13	租金递增配置器	—	`/contracts/:id/escalation`	★★★

P2：第三周完成

编号	页面	Flutter 路由	admin 路由	设计复杂度
14	二房东填报 Portal	—	`/sublease-portal`	★★☆
15	工单详情/审核/完工	`/workorders/:id`	`/workorders/:id`	★☆☆
16	租客管理	—	`/tenants`	★☆☆
17	用户管理/系统设置	—	`/settings`	★☆☆
18	支出录入	—	`/finance/expenses/new`	★☆☆

---

9. 协作检查清单

9.1 开始骨架开发前（Day 0）

- [ ] `flutter doctor` 确认环境就绪
- [ ] `flutter_app/pubspec.yaml` 依赖包版本锁定（flutter*bloc*、*dio*、*go*router、get_it）
- [ ] `admin/package.json` 依赖包版本锁定（element-plus、axios、pinia、vue-router）
- [ ] MockData 字段名与 ARCH.md §3 数据库字段名保持一致
- [ ] CSS 变量占位色已写入，骨架阶段用默认色

9.2 对齐日前（Day 3）

- [ ] 所有 P0 页面可在真机/浏览器中完整浏览（无崩溃）
- [ ] 截图包已导出（19 张）
- [ ] 5 段操作录屏已录制（建议用 QuickTime 录屏）
- [ ] 本文 §5.3 数据边界速查表已交给设计师

9.3 Design Token 接入前（Day 11）

- [ ] 设计师已提交完整 Token 表（颜色/字体/间距/圆角）
- [ ] P0 页面 Figma 稿已完成（含标注）
- [ ] 开发者已将 Token 填入 Material 3 ThemeData / Element Plus 主题文件
- [ ] 业务语义色（success/warning/danger/neutral）未被设计师更改语义

9.4 视觉验收（Day 14）

- [ ] 设计师在 iPhone 14 Pro 真机验收所有 P0 的 Flutter 页面
- [ ] 设计师在 1920×1080 浏览器验收所有 P0 的 admin 页面
- [ ] Redline 修正清单已提交并修复
- [ ] `flutter analyze` 无 Warning（flutter_app）、`npm run lint` 无 Warning（admin）

附录：快速参考

状态色彩速查

Flutter 端（Material 3）：

```
// 使用 Theme 获取颜色，禁止 Colors.green / Color(0xFF...) 硬编码
Container(
  decoration: BoxDecoration(
    color: unitStatusColor(context, unit.status),
    borderRadius: BorderRadius.circular(4),
  ),
  child: Text(unit.name),
)
```

admin 端（Element Plus）：

```
// admin/src/utils/status.ts
export function unitStatusTagType(status: string): '' | 'success' | 'warning' | 'danger' |
'info' {
  switch (status) {
    case 'leased':        return 'success'
    case 'expiring_soon': return 'warning'
    case 'vacant':        return 'danger'
    case 'non_leasable':  return 'info'
    default:              return ''
  }
}
```

Mock 数据替换路径

当后端 API 就绪时，替换方式如下：

Flutter 端：

```
// 原来（原型阶段）—— Cubit 使用 mock
emit(DashboardState.loaded(MockDashboardData.summary));

// 替换为（接入真实 API）
final summary = await repository.fetchSummary();
emit(DashboardState.loaded(summary));
```

Admin 端：

```
// 原来（原型阶段）—— store 中使用 mock
const list = ref<Dashboard>(MockDashboardData)

// 替换为（接入真实 API）
const list = ref<Dashboard>()
async function fetchSummary() {
  list.value = await apiGet<Dashboard>(API_PATHS.DASHBOARD)
```

```
}
```

页面 Widget / Vue 组件代码**不需要任何改动**，只改数据源。

文档结束 · 如有疑问联系开发负责人